DESARROLLO FULLSTACK · UNIDAD 3 · SESIÓN DE 90 MINUTOS

Autenticación y autorización en APIs REST

Integración de JWT y control de acceso por roles en un back-end Node.js + Express



eyJhbGciOiJIUzI1Ni….eyJzdWIiOjEsImVtYW….tM38WJmr7Ij1…

header · payload · firma

Facultad de Ingeniería de Sistemas

Logro y ruta de la sesión

INICIO

LOGRO DE LA UNIDAD

Implementa una aplicación front-end que interactúa con servicios RESTful para la gestión de datos.

LOGRO DE LA SESIÓN

Implementa registro, login con JWT y protección de rutas por rol en una API REST, y la consume desde un cliente front-end.

Ruta de 90 minutos

10' **Inicio**
Analogía y saberes previos

5' **Utilidad**
Riesgos reales en APIs

45' **Transformación**
Conceptos + 5 ejemplos ejecutados

22' **Práctica**
Laboratorio guiado y reto

8' **Cierre**
Verificación y conclusiones

¿Quién eres y a dónde puedes entrar?

INICIO



AUTENTICACIÓN

El DNI en el counter

Demuestra QUIÉN eres. Se revisa una sola vez, al inicio.

En la API → login con email y contraseña



AUTORIZACIÓN

La tarjeta de embarque

Dice a QUÉ puedes acceder: vuelo, puerta, asiento, sala VIP.

En la API → rol dentro del token (admin, estudiante…)



Pregunta para el aula: si alguien imprime una tarjeta de embarque falsa con “Sala VIP”, ¿cómo lo detecta la aerolínea? ¿Cómo lo detectaría nuestra API?

Por qué importa

UTILIDAD

4 de 10

riesgos del OWASP API Security Top 10 (2023) son fallas de autenticación o autorización

Toda API que gestiona datos de personas necesita saber quién llama y qué tiene permitido hacer.

API1 — **Broken Object Level Authorization**
Un usuario lee o edita registros de otro cambiando un id en la URL.

API2 — **Broken Authentication**
Contraseñas débiles, tokens sin expiración o sin verificar la firma.

API3 — **Broken Object Property Level Authorization**
La API expone o deja modificar campos sensibles (ej. el rol).

API5 — **Broken Function Level Authorization**
Un usuario común invoca endpoints de administrador.

Autenticación vs. autorización

TRANSFORMACIÓN



Autenticación

¿Quién eres?

- Ocurre primero
- Verifica credenciales: contraseña, token, huella
- Resultado: una identidad (usuario 1, ana@…)

Si falla, la API responde: **401 Unauthorized**





Autorización

¿Qué puedes hacer?

- Ocurre después de autenticar
- Verifica permisos: rol, dueño del recurso
- Resultado: permitir o denegar la acción

Si falla, la API responde: **403 Forbidden**

Cuatro formas de autenticar una API

TRANSFORMACIÓN



Sesión + cookie

El servidor guarda la sesión y envía un id en una cookie. Stateful.

Apps web clásicas (MVC)



API Key

Una clave fija por aplicación en cada petición. No identifica personas.

Integraciones servidor-servidor



JWT (JSON Web Token)

HOY

Token firmado que viaja en cada petición. El servidor no guarda estado.

SPA en React y apps móviles



OAuth 2.0 / OpenID Connect

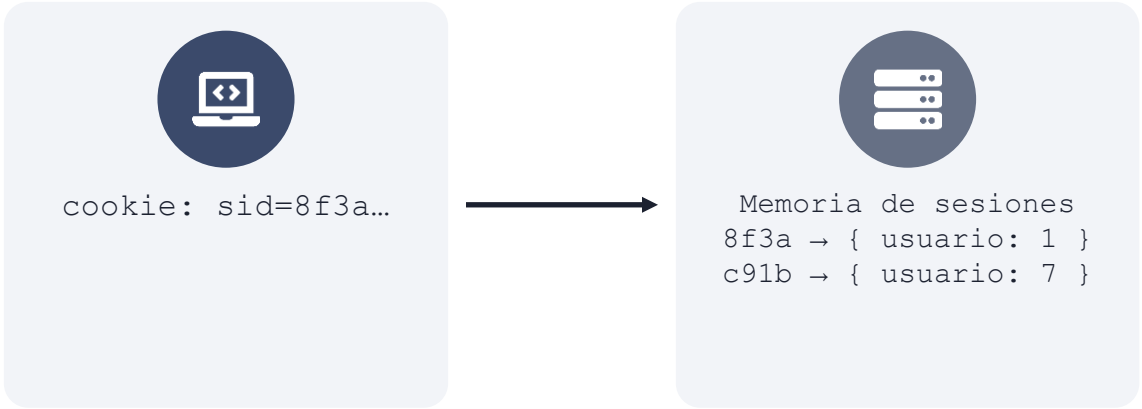
Delegas el login en un proveedor de identidad (Google, Microsoft).

“Iniciar sesión con Google”

¿Dónde vive la sesión?

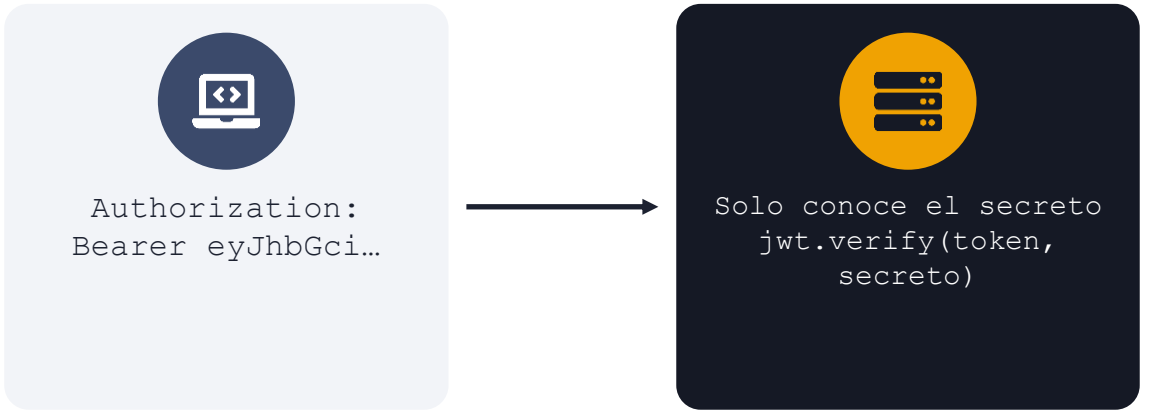
TRANSFORMACIÓN

Sesión (stateful)



- Cerrar sesión es inmediato
- El servidor guarda estado: escalar exige compartir sesiones

JWT (stateless)



- Escala fácil: cualquier servidor verifica
- No se revoca antes de expirar: usa expiraciones cortas

Nuestra elección: JWT, porque el front-end es una SPA en React que consume una API REST desacoplada.

Anatomía de un JWT

TRANSFORMACIÓN

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOjEsImVtYWlsIjoiYW5hQHVuaS5lZHUucGUiLCJyb2wiOiJlc3R1ZGlhbnRlIiwiaWF0IjoxNzkzMTI5ODA0LCJleHAiOjE3OTMxMzM0MDR9.tM38WJmr7Ij13u5FatJAMkc6c7H3Sea3hLtmEt0P9iQ
```

HEADER
Algoritmo y tipo

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

PAYLOAD
Datos (claims)

```
{
  "sub": 1,
  "email": "ana@uni.edu.pe",
  "rol": "estudiante",
  "iat": 1793129804,
  "exp": 1793133404
}
```

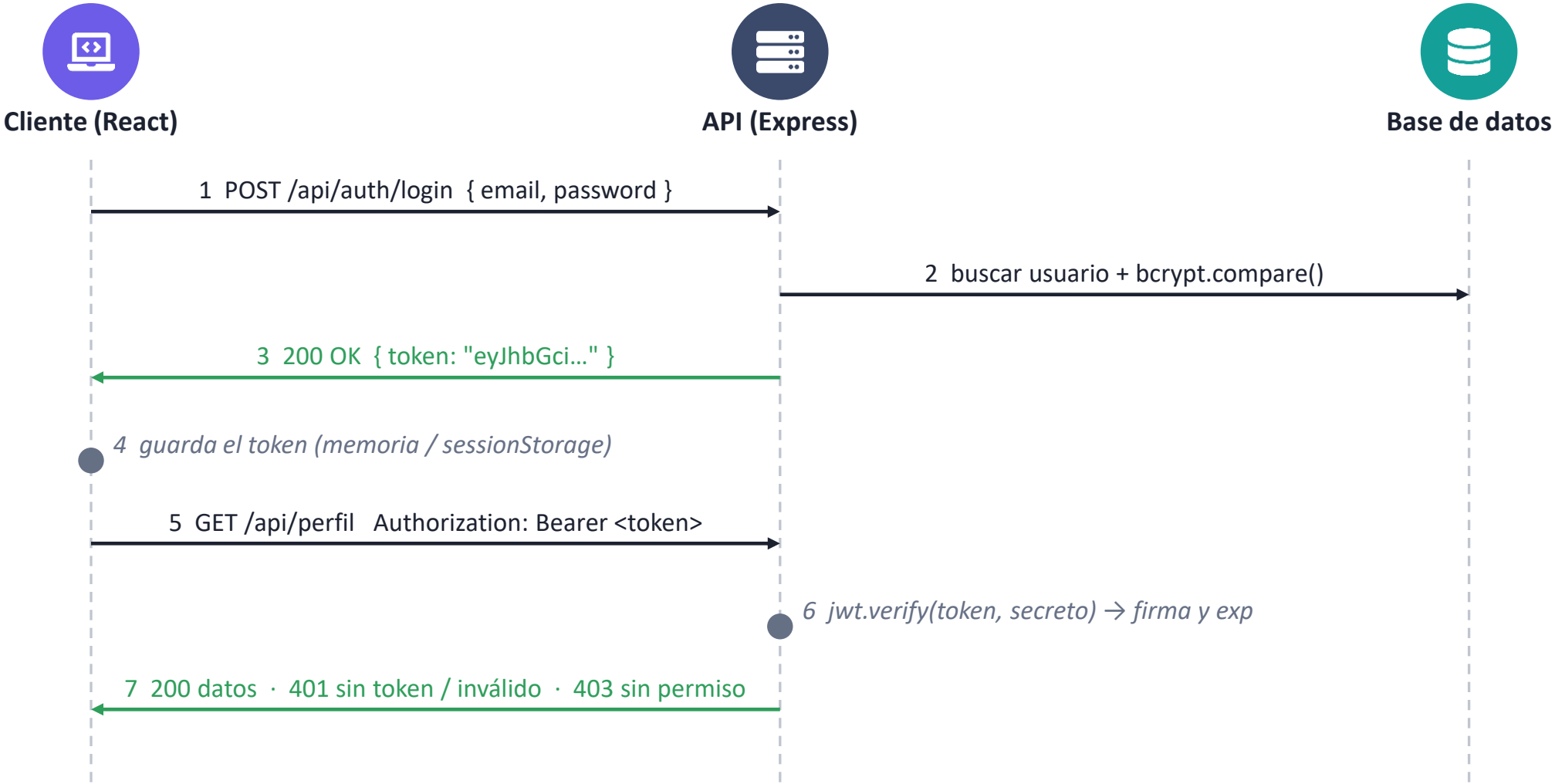
SIGNATURE
Firma

```
HMACSHA256(
  base64url(header) + "." +
  base64url(payload),
  JWT_SECRET
)
```

Base64url no es cifrado: cualquiera puede leer el payload. Nunca pongas contraseñas ni datos sensibles en él.

El flujo completo con JWT

TRANSFORMACIÓN



Guardamos el hash, no la contraseña

TRANSFORMACIÓN

Contraseña
Clave123

→ **bcrypt**
sal + 2^{10} rondas →

Se guarda en la BD (hash real)
`$2b$10$.8V5cKg6vyPIGcaaRe5XaOvyKxvTdKi6Qx6lWHDs64IUTRu9lFJN6`

$2b$ versión **10$** costo **22 caracteres** sal **resto** hash

Un solo sentido
No existe “descifrar”. Para el login se compara con bcrypt.compare().

Sal aleatoria
Dos usuarios con la misma clave obtienen hashes distintos.

Lento a propósito
El costo (10) frena los ataques de fuerza bruta.

bcrypt en Node.js

```
const bcrypt = require('bcryptjs');

// Registro: generar el hash
const hash = await bcrypt.hash('Clave123',
10);

// Login: comparar (true/false)
const ok = await bcrypt.compare(
  'Clave123', hash);
```

Proyecto de ejemplo: auth-api

TRANSFORMACIÓN

terminal

```
$ mkdir auth-api && cd auth-api
  npm init -y
  npm install express jsonwebtoken \
    bcryptjs cors
  node server.js

API escuchando en http://localhost:3000
```

express servidor HTTP y rutas

jsonwebtoken firmar y verificar JWT

bcryptjs hash de contraseñas

cors permitir al front-end React (puerto 5173)

Método	Ruta	Protección
GET	`/api/publico`	Ninguna
POST	`/api/auth/register`	Ninguna
POST	`/api/auth/login`	Ninguna · devuelve el JWT
GET	`/api/perfil`	Token (autenticación)
GET	`/api/admin/usuarios`	Token + rol admin (autorización)

Las rutas protegidas encadenan middlewares: primero autentican y luego autorizan.

Ejemplo 1 · Registro de usuario

TRANSFORMACIÓN

server.js

```
app.post('/api/auth/register', async (req, res) => {
  const { email, password, rol = 'estudiante' } = req.body;
  if (!email || !password) {
    return res.status(400)
      .json({ error: 'email y password son obligatorios' });
  }
  if (usuarios.find(u => u.email === email)) {
    return res.status(409)
      .json({ error: 'El email ya está registrado' });
  }
  // Nunca guardamos el password en texto plano
  const passwordHash = await bcrypt.hash(password, 10);
  const nuevo = { id: usuarios.length + 1, email,
                  passwordHash, rol };
  usuarios.push(nuevo);
  res.status(201).json({ id: nuevo.id, email, rol });
});
```

resultado · usuario nuevo

```
$ curl -X POST .../api/auth/register
   -d '{"email":"ana@uni.edu.pe",
        "password":"Clave123"}'

{"id":1,"email":"ana@uni.edu.pe",
 "rol":"estudiante"}
```

201 Created

resultado · email repetido

```
$ curl -X POST .../api/auth/register
   -d '{"email":"ana@uni.edu.pe",
        "password":"x"}'

{"error":"El email ya está
          registrado"}
```

409 Conflict

Ejemplo 2 · Login y emisión del JWT

TRANSFORMACIÓN

server.js

```
app.post('/api/auth/login', async (req, res) => {
  const { email, password } = req.body;
  const usuario = usuarios.find(u => u.email === email);
  const ok = usuario &&
    await bcrypt.compare(password, usuario.passwordHash);
  if (!ok) {
    return res.status(401)
      .json({ error: 'Credenciales inválidas' });
  }
  const token = jwt.sign(
    { sub: usuario.id, email: usuario.email,
      rol: usuario.rol },          // payload
    JWT_SECRET,                    // clave secreta
    { expiresIn: JWT_EXPIRES }     // ej. '1h'
  );
  res.json({ token, tipo: 'Bearer', expiraEn: JWT_EXPIRES });
});
```

resultado · contraseña errada

```
$ curl -X POST .../api/auth/login
   -d '{..."password":"mala"}'

{"error":"Credenciales inválidas"}
```

401 Unauthorized

resultado · credenciales correctas

```
$ curl -X POST .../api/auth/login
   -d '{..."password":"Clave123"}'

{"token":"eyJhbGciOiJIUzI1NiIsInR5
cCI6IkpXVCJ9.eyJzdWIiOjEsImVtYW…
…tM38WJmr7Ij13u5FatJAMkc6c7H3S",
 "tipo":"Bearer","expiraEn":"1h"}
```

200 OK

Ejemplo 3 · ¿Qué hay dentro del token?

TRANSFORMACIÓN

decodificar.js · sin usar el secreto

```
$ node -e 'const [h,p] = token.split(".");
   console.log(JSON.parse(
     Buffer.from(h, "base64url")));
   console.log(JSON.parse(
     Buffer.from(p, "base64url")))'

{ alg: 'HS256', typ: 'JWT' }
{
  sub: 1,
  email: 'ana@uni.edu.pe',
  rol: 'estudiante',
  iat: 1793129804,
  exp: 1793133404
}
```

sub	subject: id del usuario autenticado
rol	claim propio: lo usaremos para autorizar
iat	1793129804 → 27/10/2026, 14:36 (Lima)
exp	1793133404 → 27/10/2026, 15:36 · 1 hora después

Cualquiera puede LEER el token. Lo que nadie puede hacer sin el secreto es MODIFICARLO y generar una firma válida.

Ejemplo 4 · Middleware de autenticación

TRANSFORMACIÓN

server.js

```
function verificarToken(req, res, next) {
  const header = req.headers.authorization || '';    // ①
  const [tipo, token] = header.split(' ');
  if (tipo !== 'Bearer' || !token) {                 // ②
    return res.status(401).json({ error: 'Token requerido' });
  }
  try {
    req.usuario = jwt.verify(token, JWT_SECRET);     // ③
    next();                                          // ④
  } catch (err) {
    const msg = err.name === 'TokenExpiredError'
      ? 'Token expirado' : 'Token inválido';
    return res.status(401).json({ error: msg });
  }
}

app.get('/api/perfil', verificarToken, (req, res) => {
  res.json({ mensaje: 'Bienvenido', usuario: req.usuario });
});
```

① Lee el header Authorization

② Exige el formato “Bearer <token>”; si falta → 401

③ jwt.verify comprueba la firma y la expiración

④ Guarda el usuario en req.usuario y continúa con next()

Resultados: cuatro casos de autenticación

TRANSFORMACIÓN

1 · Sin token

```
$ curl .../api/perfil

{"error":"Token requerido"}
```

401 Unauthorized

2 · Token válido

```
$ curl .../api/perfil \
   -H "Authorization: Bearer $TOKEN"

{"mensaje":"Bienvenido","usuario":
 {"sub":1,"email":"ana@uni.edu.pe",
  "rol":"estudiante",...}}
```

200 OK

3 · Firma alterada

```
$ curl .../api/perfil \
   -H "Authorization: Bearer ${TOKEN%?}X"

{"error":"Token inválido"}
```

401 Unauthorized

4 · Token expirado

```
$ JWT_EXPIRES=2s node server.js
  # ...3 segundos después
  curl .../api/perfil -H "Bearer $T"

{"error":"Token expirado"}
```

401 Unauthorized

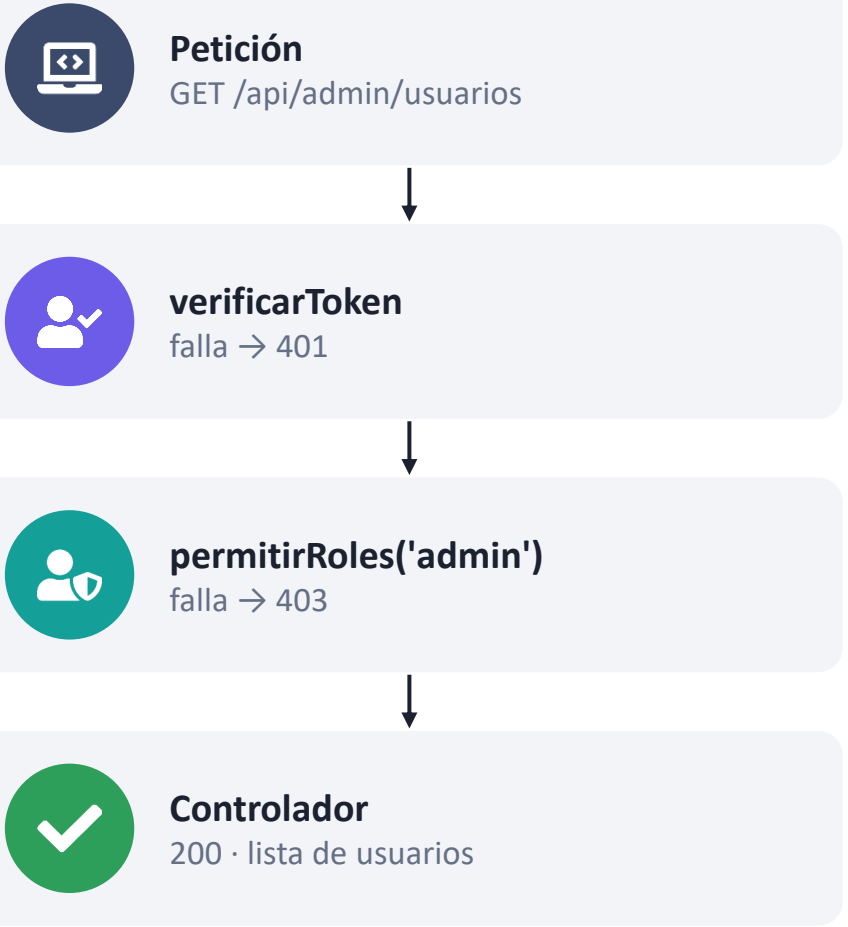
Ejemplo 5 · Autorización por roles

TRANSFORMACIÓN

server.js

```
function permitirRoles(...rolesPermitidos) {
  return (req, res, next) => {
    if (!rolesPermitidos.includes(req.usuario.rol)) {
      return res.status(403).json({
        error: `Acceso denegado para rol '${req.usuario.rol}'`
      });
    }
    next();
  };
}

app.get('/api/admin/usuarios',
  verificarToken,            // 1. ¿quién eres?
  permitirRoles('admin'),    // 2. ¿puedes entrar?
  (req, res) => {
    res.json(usuarios.map(({ passwordHash, ...u }) => u));
  });
```



Resultados: tres casos de autorización

TRANSFORMACIÓN

1 · Estudiante pide /admin

```
$ curl .../api/admin/usuarios \
    -H "Bearer $TOKEN_ANA"

{"error":"Acceso denegado
  para rol 'estudiante'"}
```

403 Forbidden

2 · Admin pide /admin

```
$ curl .../api/admin/usuarios \
    -H "Bearer $TOKEN_ADMIN"

[{"id":1,"email":"ana@uni.edu.pe",
  "rol":"estudiante"},
 {"id":2,"email":"admin@uni.edu.pe",
  "rol":"admin"}]
```

200 OK

3 · Payload con rol admin

```
$ # se edita el payload y se
  # conserva la firma original
  curl .../api/admin/usuarios \
    -H "Bearer $TOKEN_FALSO"

{"error":"Token inválido"}
```

401 Unauthorized



Respuesta a la pregunta del aeropuerto: la firma protege la integridad del token. Sin el secreto, no se puede fabricar un “boleto VIP”.

Consumir la API desde React

TRANSFORMACIÓN

src/api/auth.js

```
const API = 'http://localhost:3000/api';

export async function login(email, password) {
  const res = await fetch(`${API}/auth/login`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ email, password }),
  });
  if (!res.ok) throw new Error('Credenciales inválidas');
  const { token } = await res.json();
  sessionStorage.setItem('token', token);
}

export async function getPerfil() {
  const token = sessionStorage.getItem('token');
  const res = await fetch(`${API}/perfil`, {
    headers: { Authorization: `Bearer ${token}` },
  });
  if (res.status === 401) window.location.href = '/login';
  return res.json();
}
```

¿Dónde guardar el token?

- **Estado en memoria**
  Seguro, pero se pierde al recargar
- **sessionStorage**
  Dura la pestaña · expuesto a XSS
- **localStorage**
  Persiste · más expuesto a XSS
- **Cookie httpOnly**
  Recomendado en producción (+ CSRF)

Buenas prácticas de seguridad

TRANSFORMACIÓN

Secreto fuera del código

JWT_SECRET en variables de entorno (.env), largo y aleatorio. Nunca en GitHub.

Expiración corta

Access token de 15–60 min; para sesiones largas, refresh token.

HTTPS siempre

El token en el header viaja cifrado solo si la conexión es HTTPS.

Errores genéricos

“Credenciales inválidas” en vez de “el email no existe”.

Limitar intentos

Rate limiting en /login (ej. express-rate-limit) contra fuerza bruta.

Autoriza en el servidor

Ocultar un botón en React no protege nada: valida el rol en la API.

Laboratorio guiado

PRÁCTICA

1	**Levanta el proyecto** Descomprime auth-api.zip, ejecuta npm install y node server.js.	3'
2	**Registra dos usuarios** Uno con rol estudiante y otro con rol admin (POST /register).	2'
3	**Obtén los tokens** Haz login con ambos y decodifica el payload en jwt.io.	3'
4	**Prueba los 7 casos** Reproduce los resultados de las diapositivas 16 y 18 en Postman.	4'
5	**Conecta tu base de datos** Reemplaza el arreglo en memoria por la tabla usuarios de tu proyecto CRUD.	3'



15 min

en parejas

Herramientas

Node.js 18+, VS Code, Postman o Thunder Client, jwt.io

Reto: extiende la API

PRÁCTICA

Reto 1 **Nuevo rol docente**

Crea GET /api/notas accesible para 'docente' y 'admin' usando permitirRoles('docente','admin').

Reto 2 **Solo mis datos**

Crea GET /api/notas/:alumnoId: un estudiante solo puede ver SUS notas (compara req.usuario.sub con el parámetro).

Reto 3 **Cambio de rol seguro**

Crea PUT /api/admin/usuarios/:id/rol solo para admin. El registro no debe permitir elegir rol admin.

Criterio	Pts
Rutas protegidas con el middleware correcto	5
Códigos 401 / 403 bien usados	4
Verifica dueño del recurso (reto 2)	5
Secreto en .env y expiración configurada	3
Evidencias de pruebas (capturas Postman)	3
Total	**20**

Verifica lo aprendido

CIERRE

1. Un cliente llama a /api/perfil sin el header Authorization. ¿Qué código HTTP debe recibir?
2. Un estudiante autenticado intenta entrar a /api/admin/usuarios. ¿401 o 403? ¿Por qué?
3. ¿El payload de un JWT está cifrado? ¿Qué datos nunca deberías poner ahí?
4. ¿Qué sucede si alguien cambia "rol":"estudiante" por "rol":"admin" dentro del token?
5. ¿Por qué guardamos un hash bcrypt y no la contraseña, ni siquiera cifrada?

Lo que nos llevamos



Autenticar ≠ autorizar

Primero ¿quién eres? (401), después ¿qué puedes hacer? (403).



El JWT viaja firmado

Se lee fácilmente, pero no se puede alterar sin el secreto.



La seguridad vive en el back-end

Hash con bcrypt, middlewares encadenados y roles verificados en la API.



PRÓXIMA SESIÓN

Seguridad en dos pasos (2FA) para la API: códigos TOTP con Google Authenticator